Date: 02-04-2026

Functions in Javascript:

Functions are the main “building blocks” of the program. They allow the code to be called many times without repetition.

Function Declaration:

```
function showMessage() {
  alert( 'Hello everyone!' );
}
```

Local Variables:

A variable declared inside a function is only visible inside that function.

For example:

```
function showMessage() {
  let message = "Hello, I'm JavaScript!"; // local variable
  alert( message );
}
showMessage(); // Hello, I'm JavaScript!


alert( message ); // <-- Error! The variable is local to the function
```

Outer Variables:

A function can access an outer variable as well

The function has full access to the outer variable. It can modify it as well.

The outer variable is only used if there’s no local one.

If a same-named variable is declared inside the function then it *shadows* the outer one. For instance, in the code below the function uses the local userName. The outer one is ignored

```
let userName = 'John';


function showMessage() {
  let userName = "Bob"; // declare a local variable


  let message = 'Hello, ' + userName; // Bob
  alert(message);
}
```

```
// the function will create and use its own userName
showMessage();


alert( userName ); // John, unchanged, the function did not access the outer variable
```

Variables declared outside of any function, such as the outer userName in the code above, are called *global*.

Global variables are visible from any function (unless shadowed by locals).

It's a good practice to minimize the use of global variables. Modern code has few or no globals. Most variables reside in their functions. Sometimes though, they can be useful to store project-level data.

Parameters:

We can pass arbitrary data to functions using parameters.

When a value is passed as a function parameter, it's also called an *argument*.

Default values:

If a function is called, but an argument is not provided, then the corresponding value becomes undefined.

Returning a value:

A function can return a value back into the calling code as the result.

```
function sum(a, b) {
  return a + b;
}
let result = sum(1, 2);
alert( result ); // 3
```

return without value immediately exit the function.

Naming a function:

Functions are actions. So their name is usually a verb. It should be brief, as accurate as possible and describe what the function does, so that someone reading the code gets an indication of what the function does.

Function expression:

There is another syntax for creating a function that is called a *Function Expression*.

It allows us to create a new function in the middle of any expression.

```
let sayHi = function() {
  alert( "Hello" );
};
```

Callback functions:

```
function ask(question, yes, no) {
  if (confirm(question)) yes()
  else no();
}
function showOk() {
  alert( "You agreed." );
}
function showCancel() {
  alert( "You canceled the execution." );
}
// usage: functions showOk, showCancel are passed as arguments to ask
ask("Do you agree?", showOk, showCancel);
```

The arguments showOk and showCancel of ask are called *callback functions* or just *callbacks*.

A function is a value representing an “action”

Regular values like strings or numbers represent the *data*.

A function can be perceived as an *action*.

We can pass it between variables and run when we want.

- Functions are values. They can be assigned, copied or declared in any place of the code.
- If the function is declared as a separate statement in the main code flow, that’s called a “Function Declaration”.
- If the function is created as a part of an expression, it’s called a “Function Expression”.
- Function Declarations are processed before the code block is executed. They are visible everywhere in the block.
- Function Expressions are created when the execution flow reaches them.

In most cases when we need to declare a function, a Function Declaration is preferable, because it is visible prior to the declaration itself. That gives us more flexibility in code organization, and is usually more readable.

So we should use a Function Expression only when a Function Declaration is not fit for the task. We’ve seen a couple of examples of that in this chapter, and will see more in the future.

Arrow functions, the basics:

```
let sum = (a, b) => a + b;
```

If there are no arguments, parentheses are empty, but they must be present:

```
let sayHi = () => alert("Hello!");

sayHi();
```

Multiline arrow function:

```
let sum = (a, b) => {  // the curly brace opens a multiline function

  let result = a + b;

  return result; // if we use curly braces, then we need an explicit "return"

};

alert( sum(1, 2) ); // 3
```

For multiline arrow functions we need to use curly braces if we are using curly braces then we need to add a explicit return statement.

Arrow functions are handy for simple actions, especially for one-liners. They come in two types:

1. Without curly braces: (...args) => expression – the right side is an expression: the function evaluates it and returns the result. Parentheses can be omitted, if there's only a single argument, e.g. n => n*2.
2. With curly braces: (...args) => { body } – brackets allow us to write multiple statements inside the function, but we need an explicit return to return something.

Debugging in the browser:

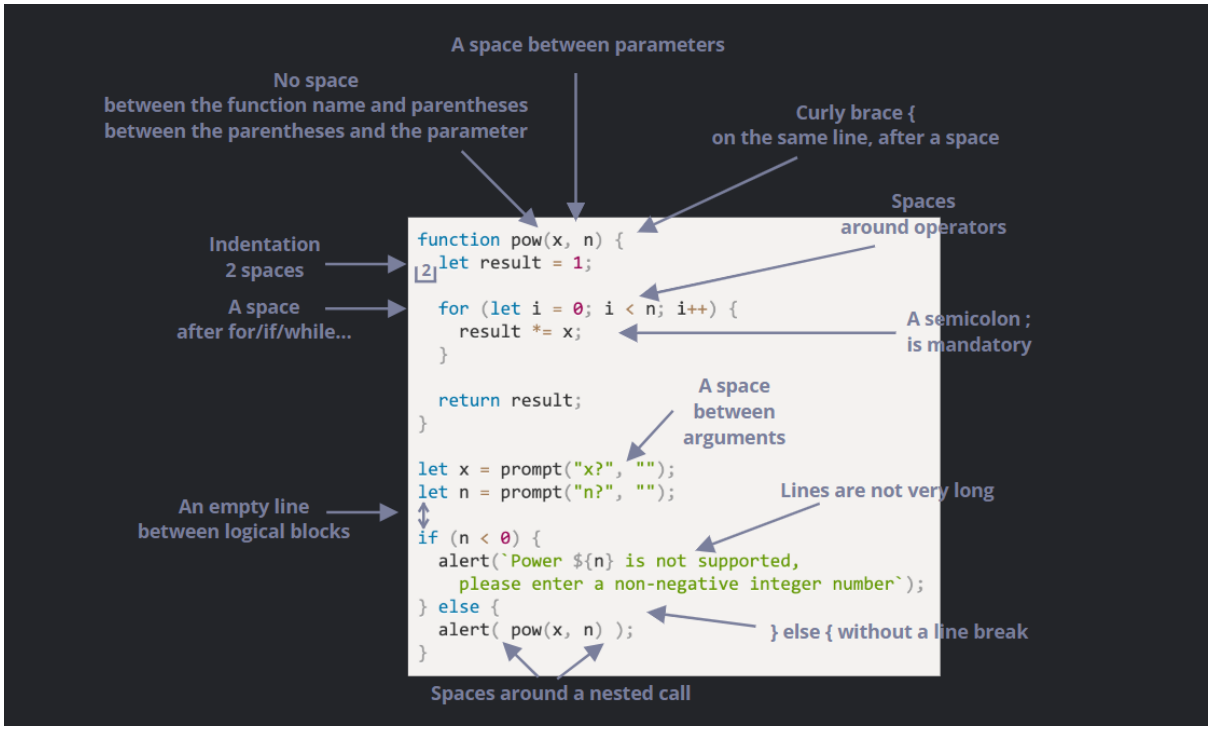
The Sources panel has 3 parts:

1. The **File Navigator** pane lists HTML, JavaScript, CSS and other files, including images that are attached to the page. Chrome extensions may appear here too.
2. The **Code Editor** pane shows the source code.
3. The **JavaScript Debugging** pane is for debugging, we'll explore it soon.

As we can see, there are three main ways to pause a script:

1. A breakpoint.
2. The debugger statements.
3. An error (if dev tools are open and the button pause is "on").

Coding style:



to increase the readability of our code.

When we think about writing “better” code, the questions we should ask ourselves are: “What makes the code more readable and easier to understand?” and “What can help us avoid errors?” These are the main things to keep in mind when choosing and debating code styles.

An important sign of a good developer is comments: their presence and even their absence.

Good comments allow us to maintain the code well, come back to it after a delay and use it more effectively.

Comment this:

- Overall architecture, high-level view.
- Function usage.
- Important solutions, especially when not immediately obvious.

Avoid comments:

- That tell “how code works” and “what it does”.
- Put them in only if it’s impossible to make the code so simple and self-descriptive that it doesn’t require them.

Comments are also used for auto-documenting tools like JSDoc3: they read them and generate HTML-docs (or docs in another format).

Ninja code:

- Ninja Code is a humorous concept that shows bad coding practices as if they are clever tricks, but its goal is to teach what to avoid.

- It includes writing overly short and complex code, using unclear variable names, excessive abbreviations, and vague naming.
- It also involves confusing code structures, hidden side effects in functions, and assigning multiple tasks to a single function.
- These practices make code difficult to read, debug, and maintain, especially in team environments.
- The main takeaway is to always write clean, simple, and readable code with meaningful names and clear logic instead of trying to be overly clever.

Automated testing with Mocha:

Behaviour driven development (BDD):

BDD is three things in one: tests AND documentation AND examples

The development flow:

The flow of development usually looks like this:

1. An initial spec is written, with tests for the most basic functionality.
2. An initial implementation is created.
3. To check whether it works, we run the testing framework Mocha that runs the spec. While the functionality is not complete, errors are displayed. We make corrections until everything works.
4. Now we have a working initial implementation with tests.
5. We add more use cases to the spec, probably not yet supported by the implementations. Tests start to fail.
6. Go to 3, update the implementation till tests give no errors.
7. Repeat steps 3-6 till the functionality is ready.

The spec in action:

The javascript libraries used for tests:

- Mocha – the core framework: it provides common testing functions including **describe** and **it** and the main function that runs tests.
- Chai – the library with many assertions. It allows to use a lot of different assertions, for now we need only **assert.equal**.
- Sinon – a library to spy over functions, emulate built-in functions and more, we'll need it much later.

```
<!DOCTYPE html>

<html>

<head>

  <!-- add mocha css, to show results -->
```

```
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/mocha/3.2.0/mocha.css">
  <!-- add mocha framework code -->
  <script src="https://cdnjs.cloudflare.com/ajax/libs/mocha/3.2.0/mocha.js"></script>
  <script>
    mocha.setup('bdd'); // minimal setup
  </script>
  <!-- add chai -->
  <script src="https://cdnjs.cloudflare.com/ajax/libs/chai/3.5.0/chai.js"></script>
  <script>
    // chai has a lot of stuff, let's make assert global
    let assert = chai.assert;
  </script>
</head>
<body>
  <script>
    function pow(x, n) {
      /* function code is to be written, empty now */
    }
  </script>
  <!-- the script with tests (describe, it...) -->
  <script src="test.js"></script>
  <!-- the element with id="mocha" will contain test results -->
  <div id="mocha"></div>
  <!-- run tests! -->
  <script>
    mocha.run();
  </script>
</body>
</html>
```

The page can be divided into five parts:

1. The <head> – add third-party libraries and styles for tests.
2. The <script> with the function to test, in our case – with the code for pow.
3. The tests – in our case an external script test.js that has describe("pow", ...) from above.
4. The HTML element will be used by Mocha to output results.
5. The tests are started by the command mocha.run().

Multiple Test Cases can be written to check different scenarios like negative numbers or zero.
Example:

```
it("adds negative numbers", function() {
  assert.equal(sum(-1, -2), -3);
});

it("adds zero", function() {
  assert.equal(sum(5, 0), 5);
});
```

- Improves Code Quality by ensuring functions behave correctly for all inputs and helps detect bugs early.

Hooks in Mocha like before, after, beforeEach, and afterEach are used to run setup or cleanup code.
Example:

```
beforeEach(function() {
  console.log("Running test...");
});
```